AutomationScheduler coordinates five tasks described in Chapter 3. Their delays, cron expressions, and time zone are read from app.scheduler properties. Per-item exception handling prevents one failed candidate or matching record from stopping an entire scan, while logs record the processing outcome.

4.8.2 Auto-Apply

AutoApplyService.runForPolicy resolves the policy owner, Candidate, and default CV and requires SCORING_DONE. It reads the top 20 Matches, considers active Jobs at or above the configured threshold, skips existing Candidate–Job applications, and creates at most three automatic applications per run. Database uniqueness is the final concurrency guard. Each created record is marked automatic, audited with the Matching and score, and followed by Candidate and Recruiter notification requests.

The current auto-apply implementation checks the supplied policy's threshold and enablement is handled by the caller selecting enabled policies. It does not independently evaluate every notification control such as quiet hours or email quotas before creating an application. Those settings primarily affect notification policy. This distinction must remain explicit: notification gating and application authorization are related but not identical mechanisms.

4.8.3 Notification Guard and Delivery

NotificationPolicyGuard evaluates whether a message is allowed and writes delivery outcomes in independent transactions. It supports deduplication, timing, quota, cooldown, and skipped/failed reasons. After CV scoring, the backend can immediately send a high-match action email, a low-match notice, or a no-match notice; the scheduled scan remains a later safety path and deduplication prevents duplicate delivery. MailService sends asynchronously through SMTP when mail is enabled, while NoOpMailService supports local development without external delivery.

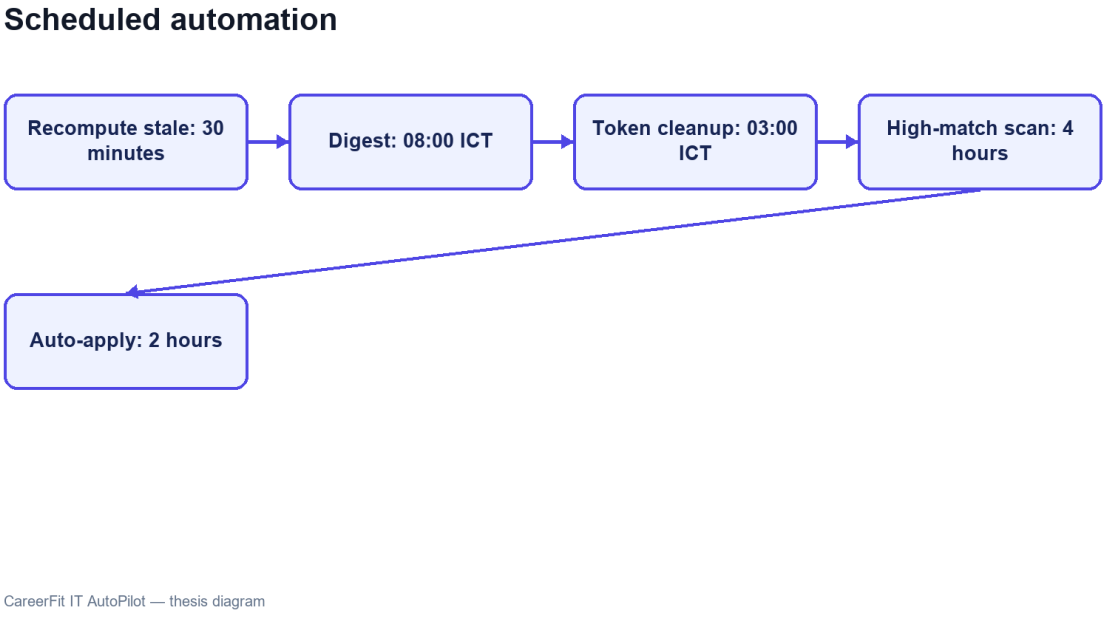


Figure 4.8. Scheduler and AutoFit execution boundaries

4.9 Email Action and Audit Implementation

EmailActionService generates a 32-character token derived from UUID text for each supported action and stores it with recipient, optional Matching, type, status, and expiry. Match email actions include GOOD_MATCH, POTENTIAL, NOT_INTERESTED, and view links. Digest messages create actions for listed Matches and unsubscribe intent. Tokens remain valid for 72 hours.

EmailActionController exposes a public confirmation and redemption flow because possession of the high-entropy token is the credential. The raw token is sent to the user but only its SHA-256 hash is persisted. GET validates the token and renders a non-mutating confirmation page; POST performs the supported action, records redemption, and rejects unknown, expired, or already-used records.

This implementation removes the earlier state-changing GET and raw-token persistence findings. The remaining production concerns are rate limiting, deployment-specific link origin, secret rotation, mail-delivery monitoring, explicit handling of expired links, and protected audit review.

Audit records are written directly through AuditLogRepository in major services. An audit entry can contain actor type/ID, action, target, result, source channel, and JSON metadata. Examples include CV failure, application submission/withdrawal, candidate invitation, status update, feedback, and auto-apply.

Direct repository calls are simple, but a centralized audit facade could standardize metadata, request IP/user-agent capture, redaction, and failure behavior.

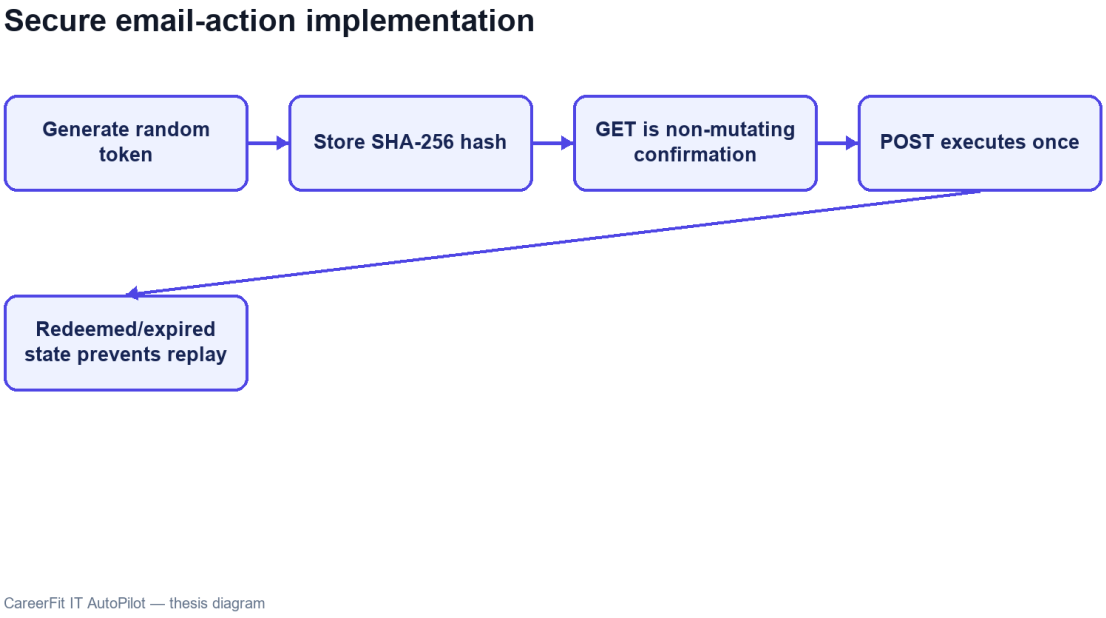


Figure 4.9. Hashed email-action token and confirm-then-POST flow

4.10 Persistence and API Implementation

JPA entities map the domain tables, while Spring Data repositories provide derived queries, pagination, locking queries, and explicit update/delete operations. Service methods use @Transactional for state changes and readOnly=true for queries where applicable. Flyway migrations V1–V15 create and harden the current schema, including hashed email-action tokens; Hibernate runs with ddl-auto=validate. This prevents application startup from silently inventing schema changes outside migration history.

Database constraints are treated as correctness boundaries rather than only documentation. Unique indexes protect email identity, default CV, Matching, Application, and imported Job hash. Check constraints restrict roles, labels, statuses, source channels, and salary modes. Version columns support optimistic concurrency on mutable core entities. JSONB stores vectors and variable metadata, while relational foreign keys preserve ownership and lifecycle relationships.

Controllers return shared API envelopes and DTOs instead of exposing entities. Validation annotations reject malformed requests near the API boundary, while service exceptions represent not found, forbidden, bad request, and conflict cases. OpenAPI is

generated through springdoc. Public list endpoints and authenticated workspaces use pagination or bounded top-result queries to avoid unbounded payloads.

Table 4.5. Representative API-to-service mapping

API operation	Controller/service responsibility	Persistence effect
POST /api/cv/upload	Validate source, create CV, start ingestion	CV, file metadata, vectors, Matchings, audit
GET /api/matches/me/cards	Resolve Candidate/default CV and map ranked results	Read-only
POST /api/matches/{id}/feedback	Upsert feedback and trigger Rocchio	Feedback, audit, learned Job vector, stale flags
POST /api/applications	Validate ownership/state and create application	Application and audit
PATCH /api/automation/policy	Validate and update owned policy	Automation policy and version
POST /api/automation/auto-apply/run-now	Execute the same policy-based service on demand	Automatic applications, audit, notifications
GET /api/admin/audit-logs	Filter and paginate administrative audit view	Read-only

4.11 Frontend Integration

App.tsx defines public, Candidate, Recruiter, and Administrator routes. protectedRoute checks the loaded account role, while reload restoration calls /api/auth/me and clears an invalid session. Passwordless login has a dedicated magic-link verification route. Public routes include Jobs and employer details; Candidate routes cover asynchronous CV processing, CV management, recommendations, applications, analytics, automation, and settings; Recruiter and Administrator routes provide their role-specific workspaces. These checks improve navigation, but the backend remains the security boundary.

Frontend/src/lib/api.ts centralizes HTTP access. It reads VITE_API_BASE_URL with /api as the default, attaches the bearer token from sessionStorage, selects JSON headers except for multipart FormData, unwraps the shared response envelope, and throws a normalized error for unsuccessful responses. DTO mapping functions convert backend shapes and labels into UI models. React Query hooks manage loading, refetching, caching, polling, and error state for server data.

The frontend stores the access token and account summary in sessionStorage, limiting persistence to the current browser tab. This still exposes the token to any successful cross-site scripting attack. The Content Security Policy reduces some

exposure, but production hardening should evaluate short-lived tokens, refresh rotation, and HttpOnly secure-cookie alternatives according to the deployed architecture.

API-driven pages no longer substitute mock Job records when requests fail; they expose loading, retryable error, or empty states. The July 18 integration pass connected magic-link completion, session revalidation, CV status polling and management, the dedicated recommendation feed, and the market dashboard to backend contracts. Remaining UI gaps are limited to selected recruiter drill-down, employer self-service, automation pause/resume, telemetry, and administrative maintenance operations.

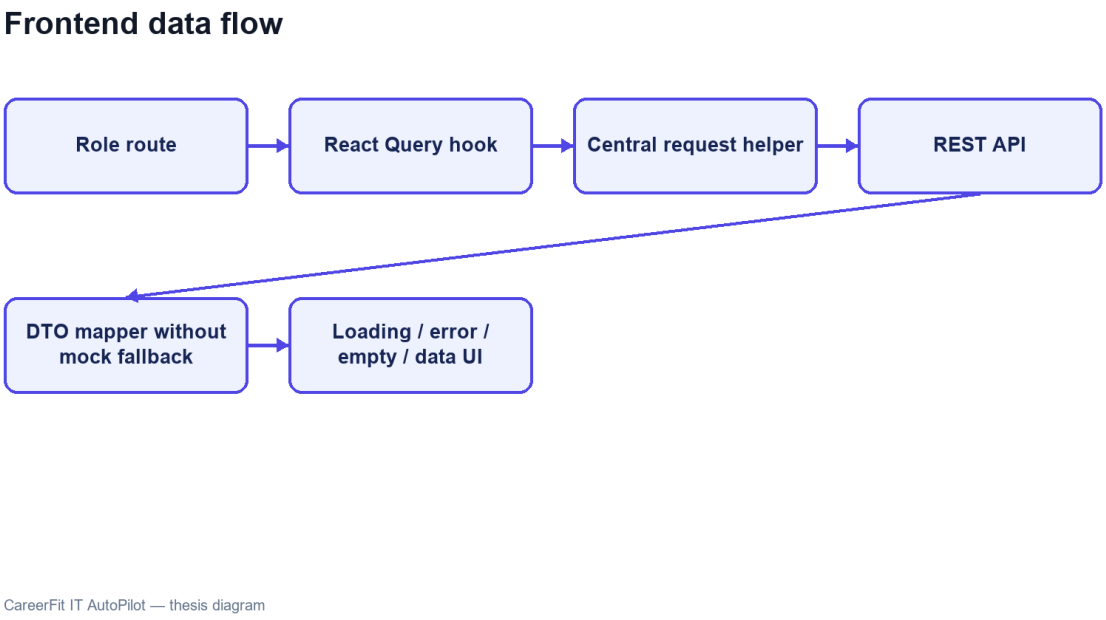
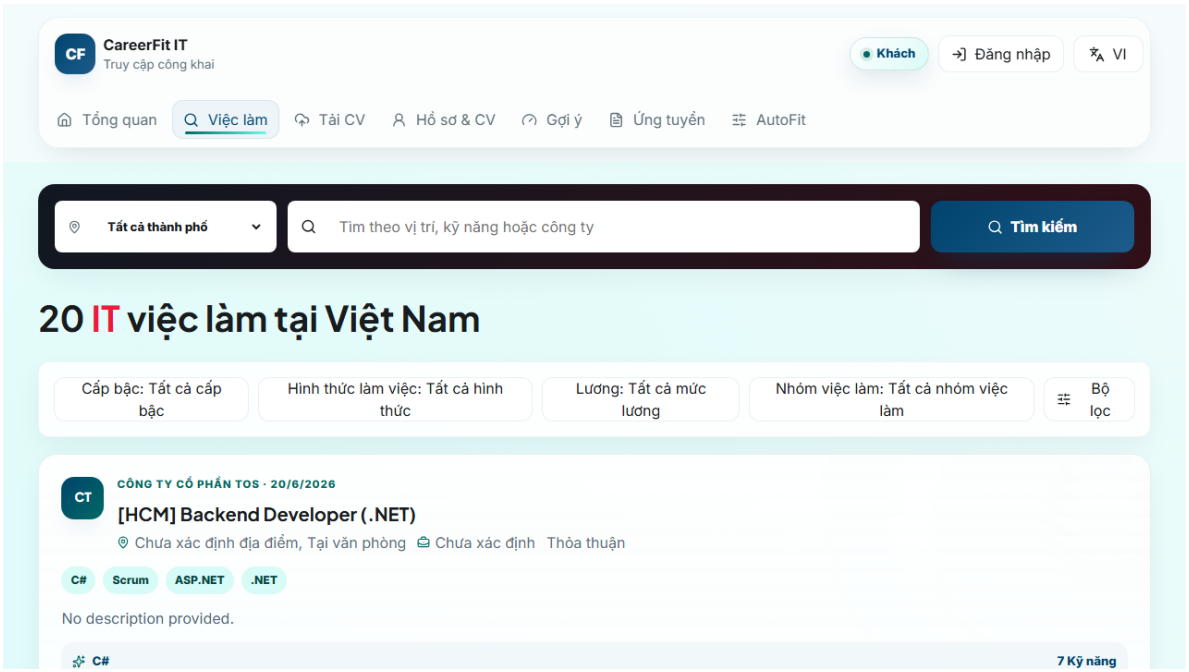
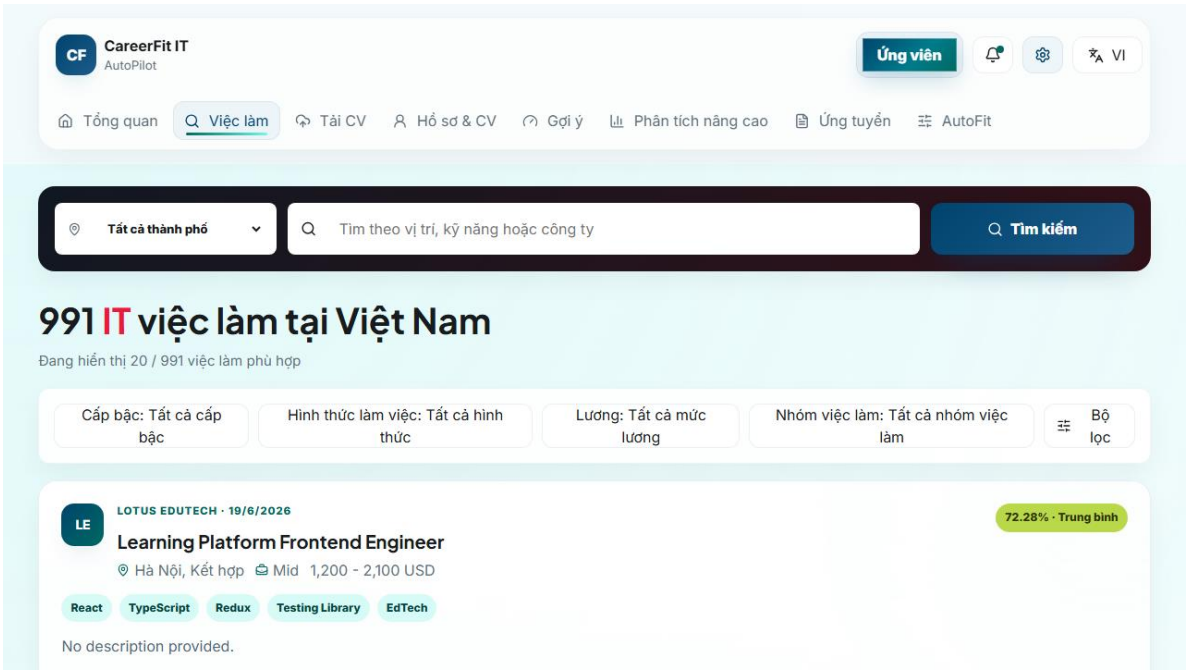


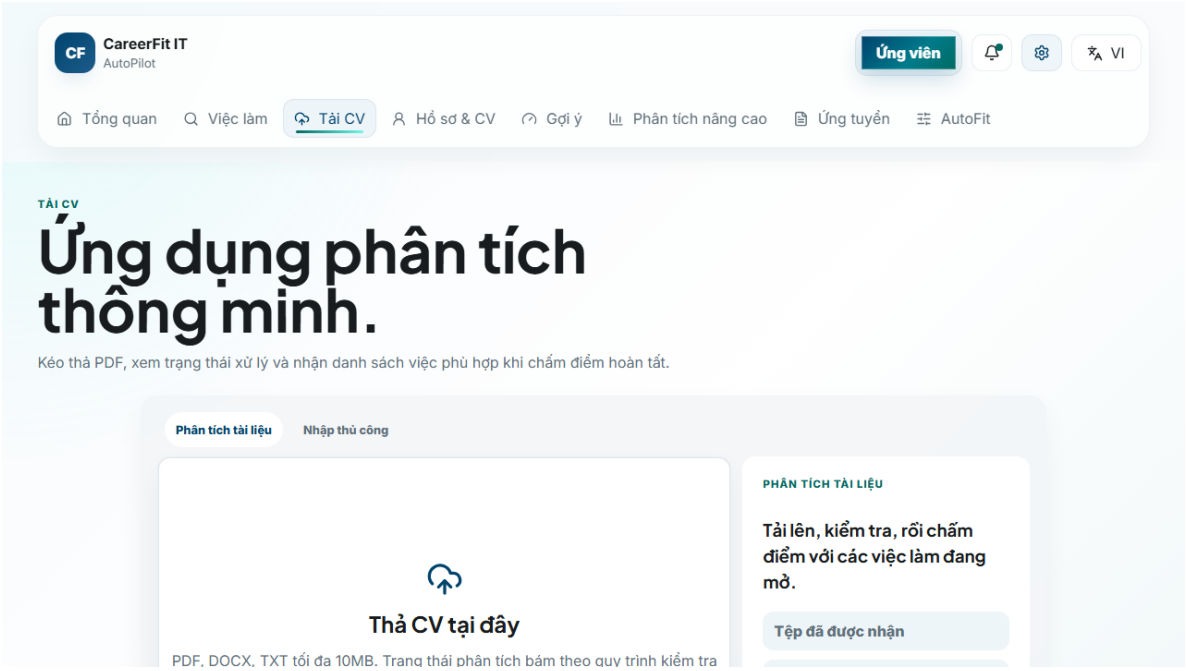
Figure 4.10. Frontend routes and API data flow



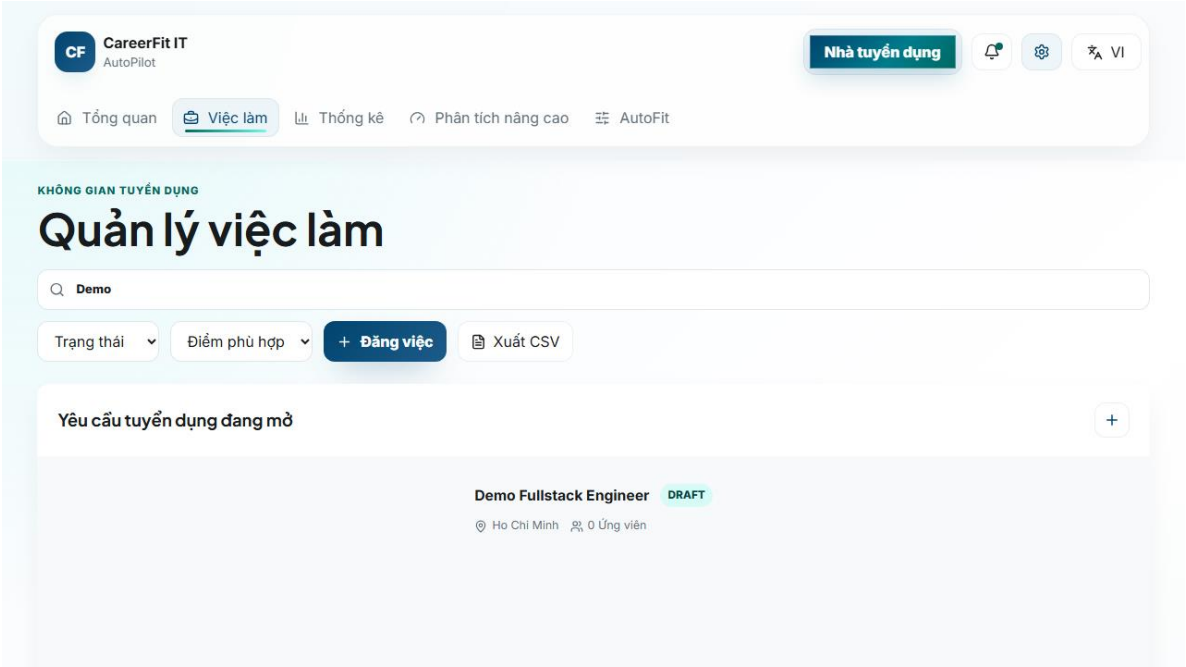
Screen 4.1. Public Job search



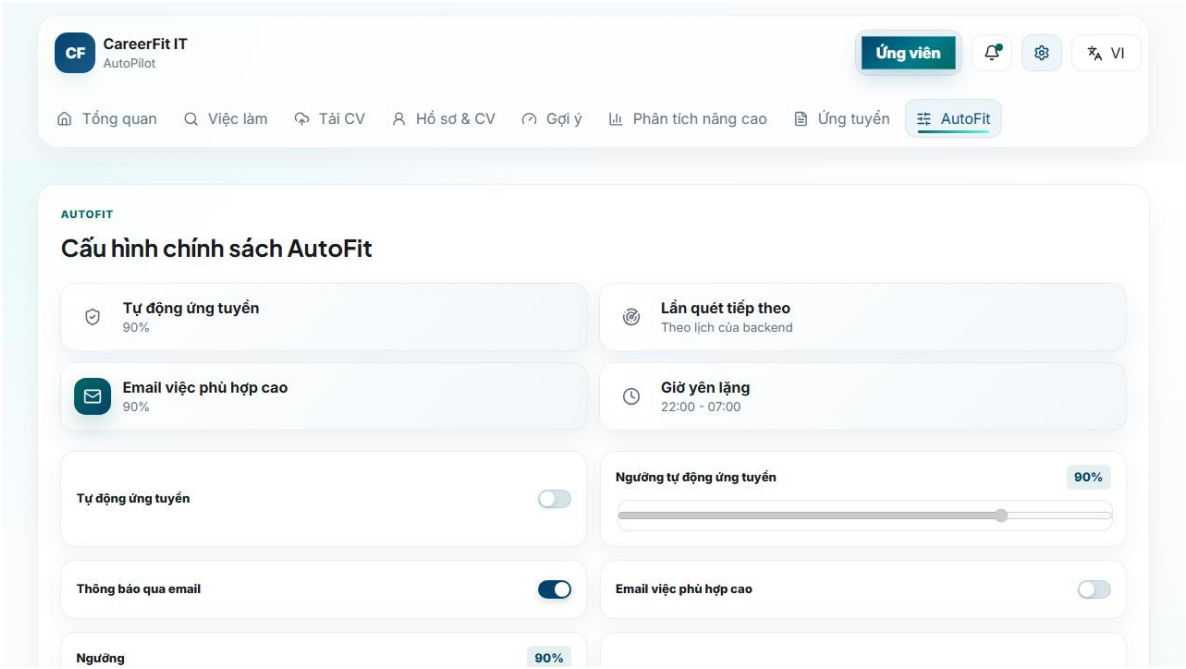
Screen 4.2. Candidate matching workspace



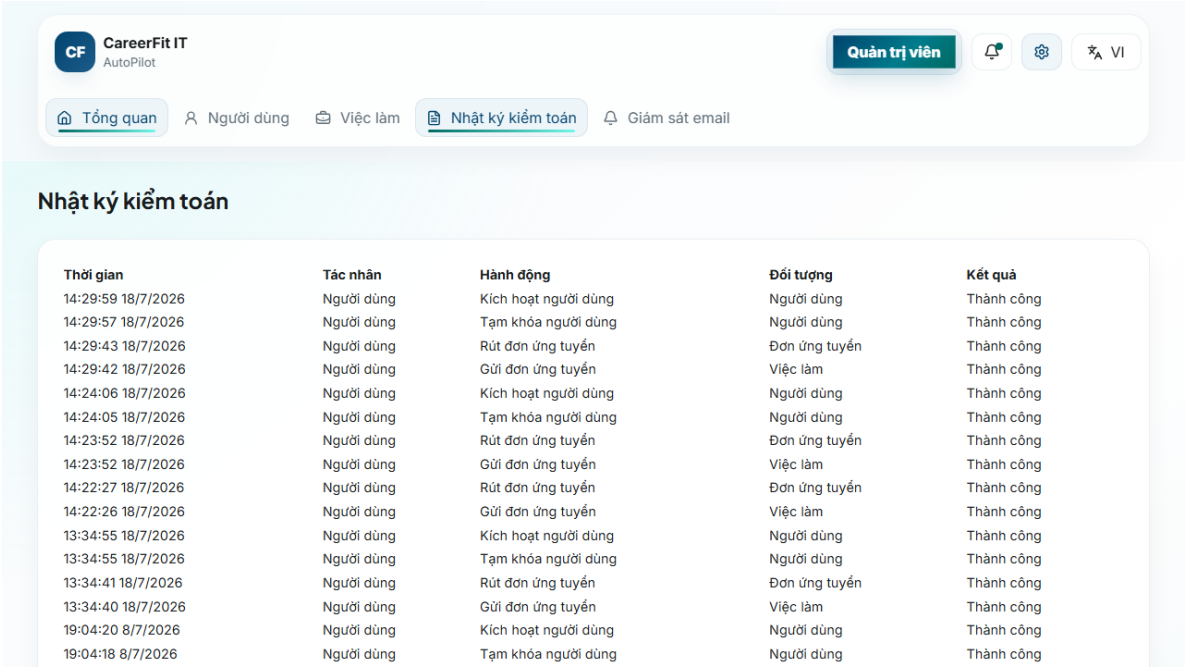
Screen 4.3. Candidate CV upload interface



Screen 4.4. Recruiter candidate workspace



Screen 4.5. Candidate AutoFit settings



Screen 4.6. Administrator audit view

4.12 Deployment and Observability Implementation

The default local runtime starts PostgreSQL through Docker Compose and can start the backend under the backend profile. Host execution connects to localhost:5433; container execution connects to postgres:5432. The backend image mounts /app/storage/cv, and environment variables override database, JWT, CORS, application URL, mail, OCR, and storage configuration. The frontend development server runs on 127.0.0.1:5173 and uses /api or a configured API base URL.

Actuator exposes health and Prometheus endpoints. Micrometer records HTTP request metrics with the application name tag and enables latency histograms. Console logs include timestamp, thread, level, request ID if present, logger, and message. These facilities provide instrumentation, but the presence of an endpoint is not equivalent to a working monitoring system; Chapter 5 must verify reachability, access restrictions, scrape behavior, and dashboard/alert evidence separately.

Maven builds the backend and can execute JUnit, Spring Security, and Testcontainers tests. The frontend build runs TypeScript checking before Vite bundling. Playwright configurations support local and production-oriented E2E runs. Build and test commands, versions, environment, failures, and generated artifacts must be recorded when Chapter 5 results are finalized.

4.13 Implementation Limitations

Table 4.6. Verified implementation limitations

Area	Current limitation	Consequence or required improvement
Token action	Hashed token storage with GET confirmation and POST execution	Add rate limiting, origin controls, secret rotation, and delivery monitoring
Text processing	Whitespace tokenization and punctuation removal	Technology aliases and Vietnamese phrases may be lost
TF-IDF corpus	Static hand-curated seed corpus; unseen terms get maximum unknown IDF	Version corpus and test domain drift/misspellings
Labels	Configured low maximum is not used as a transition; potential is primarily a boolean	Align configuration names, schema label semantics, UI mapping, and boundary tests
Scheduler	All scheduler expressions use app.scheduler properties	Add schedule-boundary and time-zone integration tests
Async consistency	Learning is registered after transaction commit	Add production retry, conflict, and failure-recovery policy

Area	Current limitation	Consequence or required improvement
Frontend session	JWT and account summary use sessionStorage	Evaluate HttpOnly cookie or refresh-token architecture and strengthen XSS controls
Frontend mapping	API-driven views no longer substitute mock Job fields	Maintain contract tests to prevent fallback data from returning
File storage	Local path or Docker volume	Add malware scanning, encryption, backup, retention, and object storage for production
Audit	Services write audit rows directly	Centralize redaction, request context, schema conventions, and integrity policy

These limitations are included because the purpose of an implementation chapter is to explain the system that exists, not an idealized version. They also provide concrete hypotheses and negative cases for Chapter 5.

4.14 Chapter Summary

This chapter explained how the design is implemented in Spring Boot, React, PostgreSQL, and Flyway. It covered security, post-commit CV and Job processing, TF-IDF scoring, Candidate feedback with Rocchio, privacy-gated portfolios, application workflows, AutoFit notifications, hardened email actions, frontend integration, and runtime monitoring. Chapter 5 evaluates these parts using refreshed test and runtime evidence.